DSR-Net: A Dual-Stream RGB–Residual CNN for AI-Generated Image Attribution, and Why It Failed under Test-Set Shift

Ahire Vrushank Ajay (G2608495L)
College of Computing and Data Science, Nanyang Technological University, Singapore
CE7454 Deep Learning for Data Science, Project 1

Code: github.com/Vrushank-Ahire/CE7454-Project1

Abstract

We classify 512×512 *JPEG images as real or as produced by one of two undisclosed generators, under a 12.3M-parameter budget with training from scratch on the 15,000 supplied images. DSR-Net is a 7.79M-parameter CNN that fuses an RGB branch with a branch fed by fixed SRM-style high-pass residuals, trained on JPEG-grid-aligned crops with JPEG re-encoding augmentation. It scores 0.991 on a held-out training split and 0.979 on the development set, but 0.484 on the hidden test set, where it predicts Model B for only 21 of 1,500 images. We report this gap as the main result, separate what the prediction files prove from what they cannot, and show that our validation protocol measured only in-distribution accuracy.*

1. Problem and data

Each image has a label $y \in \{0, 1, 2\}$ (real, Model A, Model B), so the task joins real/fake detection with source attribution [14]. The rules forbid pretrained weights, external data, ensembles, test-time augmentation and distillation. The training set has 5,000 images per class; dev (900) and test (1,500) labels are hidden and scored on CodaBench.

Supplied samples: observed vs. inferred. Figure 1 shows the twelve supplied samples. We *observe* that: (i) all twelve are 512×512 baseline JPEGs with identical IJG quality-95 tables and 4:2:0 chroma subsampling; (ii) within a scene, the three images share subject, objects and layout, but the generated ones change pose and viewpoint and are not pixel-aligned with the photograph; (iii) the photographs carry phone-filter vignetting and grading, while the generated images look cleaner and more saturated; (iv) one real file keeps an `AppleMark` JPEG comment from its source. We *infer*, without being able to verify, that the generated images came from text-to-image synthesis conditioned on a description of the photograph, and that all classes were then resized and re-encoded to a common format. Scene semantics therefore carry little label information. A detector has to rely on low-level traces and on each class's processing history, which is where shortcut cues and fragility to post-processing are known risks [5, 7]. DSR-Net reads decoded pixels only, so metadata such as (iv) is unused.

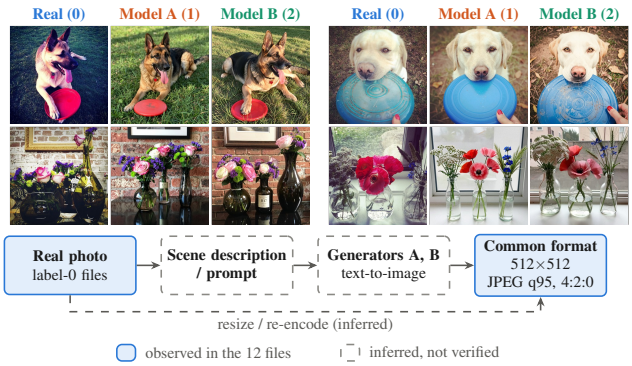


Figure 1. The twelve supplied training samples (four scenes × three classes) and our reading of the generation pipeline. Blue solid boxes are observed in the files; grey dashed boxes and arrows are inferred.

2. Related work

Wang *et al*. [12] showed that a ResNet-50 trained on one GAN with blur and JPEG augmentation transfers to other CNN generators. Frank *et al*. [3] traced detectability to spectral up-sampling artefacts, and Tan *et al*. [11] model local up-sampling correlations directly. Yu *et al*. [14] attribute images to individual GANs through learned fingerprints, the analogue of our A-vs-B decision. Corvi *et al*. [2] found that GAN-trained detectors transfer poorly to diffusion models and degrade on resized and compressed images. Ojha *et al*. [10] generalise better with frozen CLIP features, which the rules exclude here. Spatial rich models [4] suppress content with fixed high-pass filters and truncate the residuals. Zhou *et al*. [15] pair an RGB stream with an SRM stream for manipulation detection, and Bayar and Stamm [1] learn such filters with a constrained layer. DSR-Net is a compact,

from-scratch classification version of the RGB+SRM design of [15], not a new principle. On bias and shift, Gragnaniello *et al*. [6] showed that resizing and compression in training and testing govern how well detectors generalise. GenImage [16] tests on unseen generators and degraded images, and Grommelt *et al*. [7] found that GenImage detectors partly learn JPEG format and image size. Removing these biases raised cross-generator accuracy by about 11 points.

3. Method

Architecture (Fig. 2). The input is RGB in $[0, 1]$, with no mean/std normalisation. The *forensic branch* filters each colour channel with three fixed 5×5 kernels: the zero-padded 3×3 second-order "square" kernel ($/4$), the KV kernel ($/12$) [4] and a horizontal first difference $[1, -1]$. The nine residual maps are clamped to ±3 and are not trained; a 3×3 convolution ($9\to32$, GroupNorm, SiLU) follows. The *RGB branch* applies a parallel 3×3 convolution ($3\to32$), which keeps colour statistics such as the grading in Fig. 1. Both branches stay at full resolution and are fused early (concatenation, then 3×3 $64\to64$), so the trunk can relate residual and colour features at fine scale. Four stages of two residual units [8] (stride 2 in the first unit) widen to 64/128/256/384 channels. Global average pooling, dropout 0.3 and a linear layer give three logits. GroupNorm [13] replaces BatchNorm because inference runs one full image at a time, at a different size from the training crops.

Rationale and one caveat. High-pass residuals suppress scene content, which here is shared across classes, and expose sensor noise and synthesis traces [4, 15]. No downsampling before stage 1 preserves high frequencies [3]. With only 15k images, fixed filters gave a stable input. Caveat: the filters act on $[0, 1]$ values rather than 8-bit values, so the ±3 clamp almost never fires and the residuals are effectively untruncated, unlike [4].

Training and inference. A random 5% of the training set (750 images, seed 42) is held out; the remaining 14,250 are used for training. With probability 0.8 the *full* image is re-encoded as JPEG at quality $q \sim \mathcal{U}[75, 98]$. A 256^2 crop is then taken at offsets that are multiples of 8, so it stays on the JPEG block grid, and flipped with probability 0.5. Optimisation uses cross-entropy, AdamW [9] (learning rate 10^{-3}, weight decay 10^{-4}, batch 32) and per-step cosine decay over 30 epochs (13,350 steps), in FP32 on one A100 (20.1 min). Validation uses full images re-encoded once at q=91, and the best validation checkpoint is kept. Dev and test images are classified at native size in one forward pass, with no test-time augmentation or ensembling.

4. Results

Learning dynamics (Fig. 3). For 14 epochs validation accuracy stays between 0.32 and 0.57, and predictions repeatedly collapse onto one or two classes (578/172/0 at epoch 1, 261/0/489 at epoch 4). Between epochs 14 and 15 it jumps from 0.528 to 0.832, while the training loss drops from 0.853 to 0.776 and then to 0.460 at epoch 16. It peaks at 0.9907 at epoch 24, the submitted checkpoint, and training-crop accuracy ends at 0.947.

Split	n	Acc.	Predicted real / A / B
Internal val (q=91)	750	0.991	220 / 256 / 274
Dev (CodaBench)	900	0.979	258 / 342 / 300
Test (CodaBench)	1,500	0.484	1,011 / 468 / 21

Table 1. The single submitted model. Counts come from the log (val) and the submitted JSON files. Dev and test labels are hidden, so no confusion matrices are possible.

Scores (Tab. 1). The internal split and dev agree (0.991 vs. 0.979), and dev predictions are spread over all classes. On test, accuracy is 0.484 and predictions are 67.4% real, 31.2% Model A and 1.4% Model B.

5. Analysis: why 0.484 after 0.979?

Evidence (from the files). E1 (Fig. 4). Model B predictions fall from 33.3% (dev) to 1.4% (test), and real predictions rise from 28.7% to 67.4%. **E2.** The shift covers the whole test set: each of the ten consecutive 150-file blocks has 0–4 Model B predictions. **E3.** At most 21 Model B images can be classified correctly, so at least 705 of the 726 correct test predictions come from real and Model A images. **E4.** If the 0.979 dev score belongs to the submitted dev file, the dev labels are not uniform: 258 real predictions against 300 real images would cap accuracy at 0.953. Test balance therefore cannot be assumed. Under a balanced test set, Model B recall would be at most 4.2%. Without labels we cannot say which predictions are wrong, nor separate Model B images absorbed into "real" from a change in class priors.

Hypotheses (not verified), with the test that would decide each. H1: processing shift. Test images underwent processing absent from training and dev (resampling, a different compression chain), which erased the high-frequency cue that identifies Model B, so those images defaulted to "real". This drift towards "real" is the failure reported for processed images [2, 6]. The abrupt jump at epoch 15 also fits a decision built on one narrow cue. *Test:* accuracy of held-out training images under rescaling and blur. **H2: format shortcut.** Real and generated images reached the common format by different routes; if the network keyed on that history [7], a differently produced test set breaks it. *Test:* compare per-class and per-split quantisation and size statistics, and retrain after randomly resampling every class. **H3: generator or prompt shift.** We have no evidence either way.

Main limitation: validation was in-distribution. The held-out split came from the training distribution, and given the triplet structure it probably contained scene siblings of train-

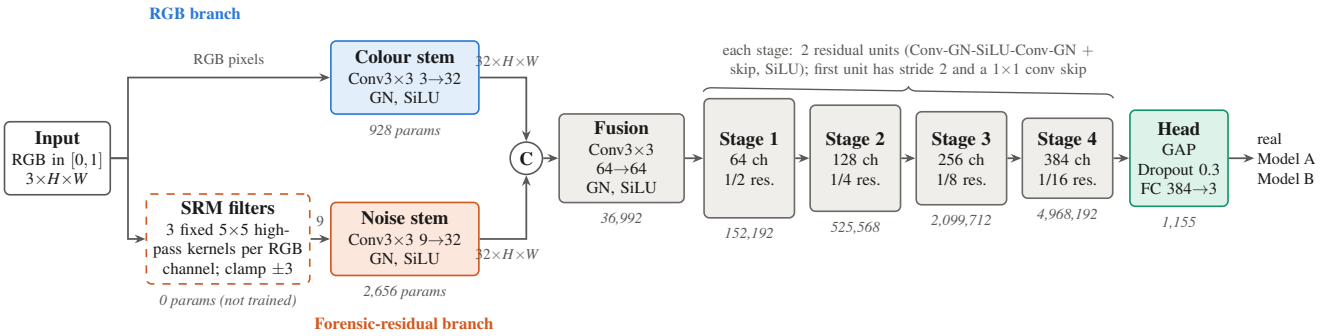


Figure 2. DSR-Net (`DualStreamForensicNet` in `aigi_jpeg.py`). The input is split into an RGB branch and a forensic-residual branch. The branches are fused at full resolution, and their combined features pass through four residual stages to a three-way classifier. Italic numbers are trainable parameters per block, computed from layer shapes; they sum to the 7,787,395 printed in the training log (limit 12.3M). The dashed SRM block is fixed. C: channel concatenation (64 channels); GN: GroupNorm (8 groups); GAP: global average pooling. Spatial size is unchanged until Stage 1 (training crops $H{=}W{=}256$; inference on full images).

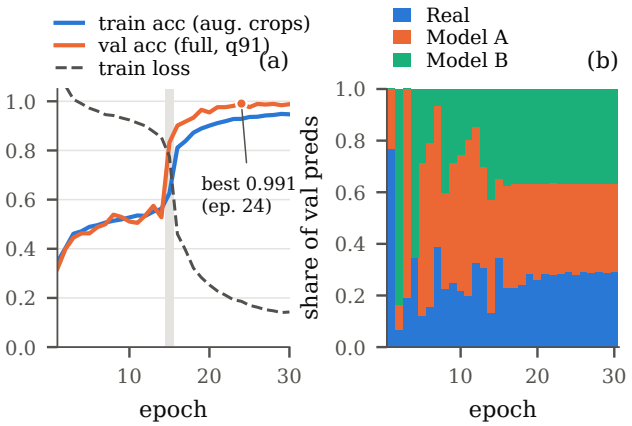


Figure 3. From the job log. (a) Training loss, training accuracy on augmented crops and validation accuracy (750 images, $q{=}91$); the grey band marks the jump at epoch 15. (b) Predicted class shares on the validation split per epoch.

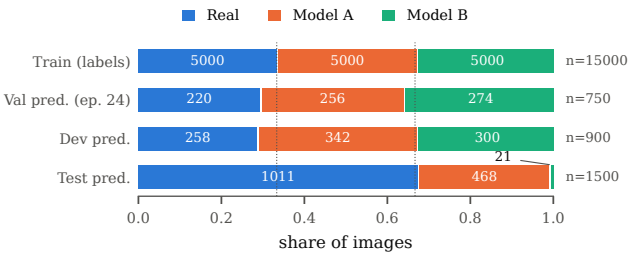


Figure 4. Class shares: training labels, and predictions on validation (epoch 24), dev and test. Dotted lines mark the 1/3 and 2/3 boundaries. Only predictions are shown for dev and test, because their labels are hidden.

ing images. Its only difference from training was one JPEG quality inside the augmentation range. Dev turned out to be in-distribution too (0.991 vs. 0.979). Model selection therefore rewarded a cue whose robustness was never measured. Training-crop accuracy (0.947) is below validation accuracy, so this is a robustness failure, not ordinary overfitting. Other limitations: one seed, twelve samples for the data analysis, and no test images.

Priority improvements, all within the rules. (1) Validation under transformations held out from training (0.5–1.5× resizing, blur, noise, double JPEG), plus monitoring the predicted class mix on the unlabelled test set, which would have flagged E1 before submission. (2) Augmentation with resampling, blur and noise applied to every class [12]. (3) Ablations: RGB vs. residual vs. both, JPEG augmentation on/off, and fixed SRM vs. a constrained layer [1]. (4) Residuals on the 8-bit scale with an active truncation threshold [4].

6. Conclusion

DSR-Net, a compact RGB+SRM CNN trained from scratch, is nearly perfect on in-distribution data and fails on the hidden test set, where it almost never predicts Model B. The architecture follows established forensic designs. What failed was the protocol: every validation signal shared the training distribution, so a fragile cue looked like a good model.

References

[1] Belhassen Bayar and Matthew C. Stamm. A deep learning approach to universal image manipulation detection using a new convolutional layer. In *Proceedings of the 4th ACM Workshop on Information Hiding and Multimedia Security (IH&MMSec)*, 2016.

[2] Riccardo Corvi, Davide Cozzolino, Giada Zingarini, Giovanni Poggi, Koki Nagano, and Luisa Verdoliva. On the detection of synthetic images generated by diffusion models. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2023.

[3] Joel Frank, Thorsten Eisenhofer, Lea Schönherr, Asja Fischer, Dorothea Kolossa, and Thorsten Holz. Leveraging frequency analysis for deep fake image recognition. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.

[4] Jessica Fridrich and Jan Kodovský. Rich models for steganalysis of digital images. *IEEE Transactions on Information Forensics and Security*, 7(3):868–882, 2012.

[5] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020.

[6] Diego Gragnaniello, Davide Cozzolino, Francesco Marra, Giovanni Poggi, and Luisa Verdoliva. Are GAN generated images easy to detect? A critical analysis of the state-of-the-art. In *IEEE International Conference on Multimedia and Expo (ICME)*, 2021.

[7] Patrick Grommelt, Louis Weiss, Franz-Josef Pfreundt, and Janis Keuper. Fake or JPEG? Revealing common biases in generated image detection datasets. In *European Conference on Computer Vision Workshops (ECCVW)*, pages 80–95, 2024.

[8] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[9] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

[10] Utkarsh Ojha, Yuheng Li, and Yong Jae Lee. Towards universal fake image detectors that generalize across generative models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[11] Chuangchuang Tan, Yao Zhao, Shikui Wei, Guanghua Gu, Ping Liu, and Yunchao Wei. Rethinking the up-sampling operations in CNN-based generative network for generalizable deepfake detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 28130–28139, 2024.

[12] Sheng-Yu Wang, Oliver Wang, Richard Zhang, Andrew Owens, and Alexei A. Efros. CNN-generated images are surprisingly easy to spot... for now. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.

[13] Yuxin Wu and Kaiming He. Group normalization. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018.

[14] Ning Yu, Larry S. Davis, and Mario Fritz. Attributing fake images to GANs: Learning and analyzing GAN fingerprints. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.

[15] Peng Zhou, Xintong Han, Vlad I. Morariu, and Larry S. Davis. Learning rich features for image manipulation detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[16] Mingjian Zhu, Hanting Chen, Qiangyu Yan, Xudong Huang, Guanyu Lin, Wei Li, Zhijun Tu, Hailin Hu, Jie Hu, and Yunhe Wang. GenImage: A million-scale benchmark for detecting AI-generated image. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.